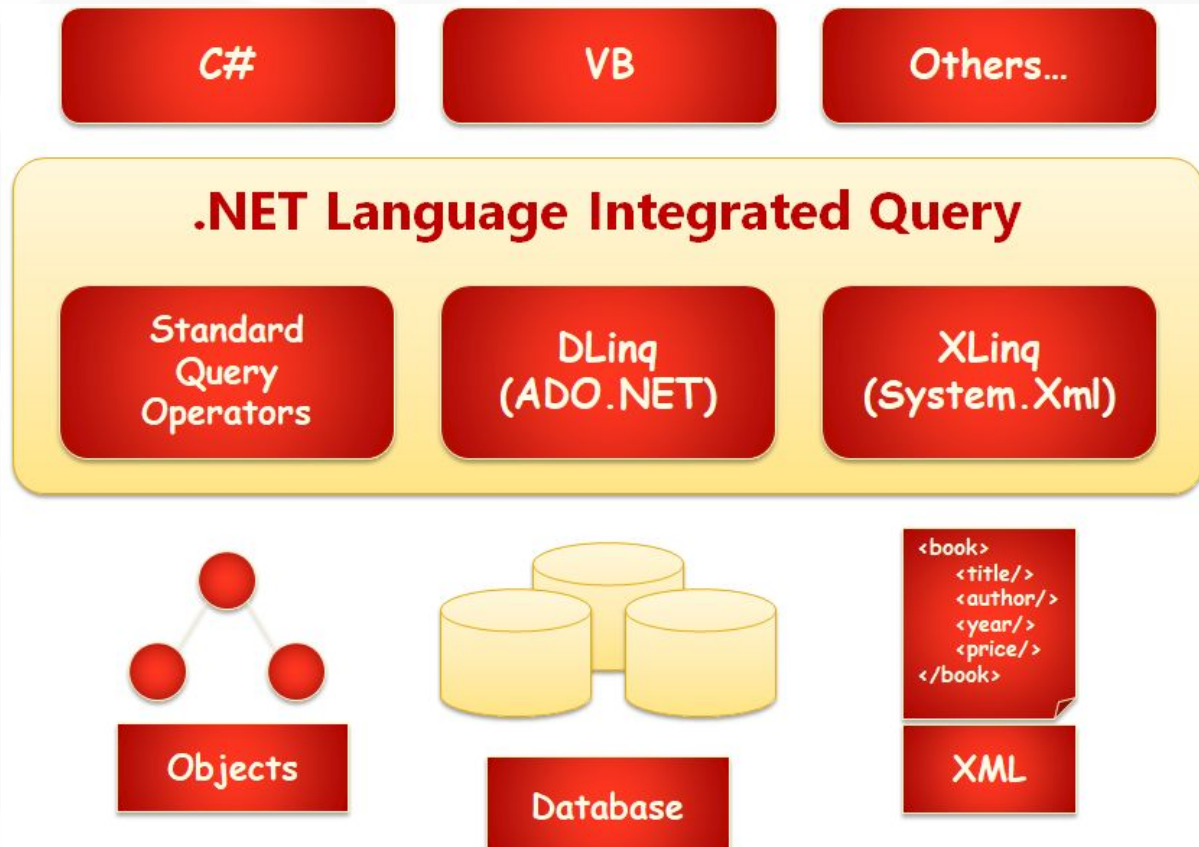


LINQ

Language INtegrated Query

Kiến trúc



Ví dụ đầu tiên - Tìm các số nhỏ hơn 5

```
var list = new List<int>() {1, 2, 3, 4, 5, 6, 7, 8, 9};
```

```
IEnumerable<int> query = from n in list
```

```
    where n < 5
```

```
    select n;
```

```
foreach (var n in query)
```

```
    Console.WriteLine(n);
```

Luyện tập query syntax / lambda syntax

```
var list = Enumerable.Range(1, 100);
```

A) Tìm các số có tận cùng là 4

B) Tìm các số chia hết cho 3

Truy vấn trên đối tượng

Chuẩn bị lớp Customer

```
public class Customer
{
    public string CustomerID { get; set; }
    public string ContactName { get; set; }
    public string City { get; set; }
}
```

Chuẩn bị hàm lấy dữ liệu

```
static List<Customer> GetCustomers()  
{  
    return new List<Customer> {  
        new Customer { CustomerID = "001", ContactName = "Maria Anders", City = "Berlin" },  
        new Customer { CustomerID = "002", ContactName = "Ana Trujillo", City = "Mexico D.F." },  
        new Customer { CustomerID = "003", ContactName = "Antonino Moreno", City = "Mexico D.F." }  
    };  
}
```

Truy vấn

```
var query = from c in GetCustomers()
where c.City == "Mexico D.F."
//where c.City.StartsWith("A")
select new { // Tạo mới một đối tượng nặc danh
    City = c.City,
    ContactName = c.ContactName
};
```

Giải thích về lớp nặc danh - Anonymous class

```
var dude = new { Name = "Bob", Age = 25 };
```



```
internal class AnonymousGeneratedTypeName  
{  
    public string Name { get; set; }  
    public int Age { get; set; }  
}
```

```
AnonymousGeneratedTypeName dude =  
    new AnonymousGeneratedTypeName {Name = "Bob", Age = 25};
```

Truy vấn lồng nhau

```
var query = from i in new List<int>(){1, 2, 3}
             from j in new List<int>() {4, 5, 6}
             where j == i * 2
             select j;

query.Dump();
```

▲ IEnumerable<Int32> (2 items) ▶

4

6

Dùng làm biến tạm

```
var deniedList = new string[] {  
    "google.com;8.8.8.8",  
    "opendns.org;203.66.172.49"  
};  
var query = from link in deniedList  
            let parts = link.Split(";")  
            select new {  
                Domain = parts[0],  
                IP = parts[1]  
            };
```

Domain	IP
google.com	8.8.8.8
opendns.org	203.66.172.49

Tách câu thành các từ

```
var sentences = new string[] { "The quick brown fox jumps over the lazy dog",  
    "She sells sea shells by the sea shore" };  
var query = from sentence in sentences  
    let words = sentence.Split(' ')  
    from word in words  
    where word.Length > 3  
    select word;
```

▲ IEnumerable<String> (8 items) ▶

quick

brown

jumps

over

lazy

sells

shells

shore

Bài tập

```
string[] sentences = {  
    "A penny saved is a penny earned",  
    "The early bird catches the worm",  
    "The second mouse gets the cheese",  
    "The pen is mightier than the sword",  
    "He who laughs last, laughs best",  
    "Teacher teaches all, teaches ill"  
};
```

Cho biết những từ bắt đầu bằng **nguyên âm**.

Đáp án

```
var earlyBirdQuery =  
    from sentence in strings  
        let words = sentence.Split(' ')  
            from word in words  
                let w = word.ToLower()  
                    where w[0] == 'a' || w[0] == 'e'  
                        || w[0] == 'i' || w[0] == 'o'  
                            || w[0] == 'u'  
                        select word;
```

Kết bảng

Có ý nghĩa như phép kết bảng trong cơ sở dữ liệu quan hệ

```
var query =  
    from c in Categories  
    join p in Products on c.CategoryID equals p.CategoryID  
    select new {c.CategoryName, p.ProductName};
```

Bài tập

Manufacturer(manufacid, manufacname)

Laptop(sku, lapname, manufacid)

Cho biết tên các laptop của hãng “Acer”

```
var manufacturers = ManufacturerDao.GetAll()
```

```
var laptops = LaptopDao.GetAll()
```

Đáp án

```
from l in laptops
joins m in manufacturers
    on {a, b, c} equals {a, b, c}
where l.lapname == "Acer"
select new {
    l.lapname
}
```

Sắp xếp kết quả - orderby

```
var query =  
    from m in typeof(string).GetMethods()  
    where m.IsStatic == true  
    orderby m.Name [descending]  
    select m.Name;
```

Bài tập

Product(SKU, Name, Price, Quantity)

TransactionLog(LogID, ProductID, SoldCount)

Cho biết tên, giá các sản phẩm bán chạy
(soldCount > 100) giảm dần theo số lượng

```
var products = Dao.GetAllProducts()
```

```
var transactions = Dao.GetAllTransactions()
```

Đáp án

```
from t in transactions
where t.SoldCount > 100
joins p in products
on p.SKU equals t.ProductId
orderby p.Quantity descending
select new {
    p.Name, p.Price
}
```

Gom nhóm kết quả

```
var query =
```

```
    from m in typeof(string).GetMethods()
```

```
    where m.IsStatic == true
```

```
    orderby m.Name [descending]
```

```
    group m by m.Name;
```

group đã bao hàm ý nghĩa **select** nên không cần **select** nữa

Group nhiều thuộc tính - Ý tưởng ban đầu

```
var query =  
    from p in Products  
    join c in Categories on p.CategoryID equals c.CategoryID  
    group p by c.CategoryID, c.CategoryName;
```



Cú pháp đúng

```
var query =  
    from p in Products  
    join c in Categories on p.CategoryID equals c.CategoryID  
    group p by new {  
        c.CategoryID, c.CategoryName  
    };
```

Lambda syntax

Where: giữ lại các phần tử thoả điều kiện

Query Syntax

```
var query = from n in list  
    where n < 3  
    select n;
```

Lambda Syntax //functional programming

```
var query = list.Where(item => item < 3 );  
var query = list.Where( (item) => { return item < 3} );
```

Select

Query Syntax

```
var query = from c in GetCustomers()  
            where c.City.StartsWith("A")  
            select new { c.City, c.ContactName };
```

Lambda Syntax

```
var query =  
    GetCustomers()  
    .Where(c1 => c1.City.StartsWith("A"))  
    .Select(c2 => new { c2.City, c2.ContactName });
```

Luyện tập

Product(SKU, Name, Quantity, Price)

ProductDAO

– List<Product> GetAll();

- Cho biết tên các mặt hàng còn trong kho
- Cho biết tên, giá các mặt hàng sắp hết (Số lượng < 10)
- Cho biết các mặt hàng có giá từ 100 đến 200

Lambda Syntax

```
var query =  
    GetCustomers()  
    .Where(c => c.City.StartsWith("A"))  
    .Select(c => new { c.City, c.ContactName });
```

Đáp án

```
var products = ProductDao.GetAll();  
products.Where( p => p.Quantity > 0)  
    .Select(p => p.Name)  
products.Where( p => p.Quantity < 10)  
    .Select(p => new {p.Name, p.Price })  
products.Where( p => 100 <= p.Price && p.Price <= 200);
```

Select với chỉ mục

```
int[] numbers = { 3, 9, 100, 4, 2, 6, 7, 1, 8 };  
var query = numbers  
    .Select((n, index) => new {index, n})  
    .Where(item => item.idx % 2 == 0);
```

Select many

Dùng phẳng hóa, biến nhiều mảng thành 1 mảng

```
public class NewItem
{
    public string Category { get; set; }
    public string Name { get; set; }
    public List<string> Ingredients { get; set; }
    public double Price { get; set; }
}
```

```
private static List<NewItem> GetNewItemslis()
{
    List<NewItem> itemsList = new List<NewItem> {
        new NewItem
        {
            Category="Icecreams", Name="Chocolate Fudge Icecream",
            Ingredients = new List<string> {"cream", "milk", "mono and diglycerides"},
            Price=10.5
        },
        new NewItem
        {
            Category="Icecreams", Name="Vanilla Icecream",
            Ingredients= new List<string> {"vanilla extract", "guar gum", "cream"},
            Price=9.80
        },
        new NewItem
        {
            Category="Icecreams", Name="Banana Split Icecream",
            Ingredients= new List<string> {"Banana", "guar gum", "cream"},
            Price=7.5
        }
    };
    return itemsList;
}
```

Nếu chỉ select thường không selectmany

```
var itemss = GetNewItemssList();  
var ingredients = itemss.Select(ing => ing.Ingredients);
```

IEnumerable<List<String>> (3 items)	
List<String> (3 items)	cream
	milk
	mono and diglycerides
List<String> (3 items)	vanilla extract
	guar gum
	cream
List<String> (3 items)	Banana
	guar gum
	cream

Sử dụng SelectMany

```
var itemss = GetNewItemssList();  
var ingredients = itemss.SelectMany(ing => ing.Ingredients);
```

▲ IEnumerable<String> (9 items)

cream

milk

mono and diglycerides

vanilla extract

guar gum

cream

Banana

guar gum

cream

Join

Query Syntax:

```
var query =  
    from c in Categories  
    join p in Products on c.CategoryID equals p.CategoryID  
    select new {c.CategoryName, p.ProductName};
```

Lambda Syntax:

```
var query = Categories.Join(  
    Products,  
    c => c.CategoryID,  
    p => p.CategoryID,  
    select new {c.CategoryName, p.ProductName}  
);
```

Join multiple fields - Query syntax

```
var query =  
    from s in ShoppingMalls  
    join h in Houses on  
    new { s.CouncilCode, s.PostCode }  
    equals new { h.CouncilCode, h.PostCode }  
    select s;
```

Join multiple fields - Lambda syntax

Lambda Syntax:

```
var query =  
    ShoppingMalls.Join(  
        Houses,  
        s => new { s.CouncilCode, s.PostCode },  
        h => new { h.CouncilCode, h.PostCode },  
        (s,h) => new {s.CouncilName, h.Name}  
    );
```

Luyện tập

TransactionDetail(tdid, lapid, soldDate)

Laptop(lid, name, importDate)

Cho biết tên các laptop của hãng acer có tháng bán được trùng với tháng nhập laptop

GroupJoin

Dựa trên kết quả join, gom nhóm lại theo key

```
public class Student{  
    public int StudentID { get; set; }  
    public string StudentName { get; set; }  
    public int StandardID { get; set; }  
}
```

```
public class Standard{  
    public int StandardID { get; set; }  
    public string StandardName { get; set; }  
}
```

```
var groupJoin = standardList.GroupJoin(studentList,  
    std => std.StandardID, //outerKeySelector  
    s => s.StandardID,     //innerKeySelector  
    (std, studentsGroup) => new // resultSelector  
    {  
        StandardID = std.StandardID,  
        StandarFullIdName = std.StandardName,  
        Students = studentsGroup,  
    }  
);
```

Kết quả GroupJoin

```
var groupJoin = standardList.GroupJoin(studentList,  
    std => std.StandardID, //outerKeySelector  
    s => s.StandardID,     //innerKeySelector  
    (std, studentsGroup) => new // resultSelector  
    {  
        StandardID = std.StandardID,  
        StandarFullIdName = std.StandardName,  
        Students = studentsGroup,  
    }  
);
```

▲ (3 items)											
StandardID	StandarFullIdName	Students									
1	Standard 1	Key= 1 ▲ Grouping<Int32,Student> (2 items) ▶ <table><tr><th>StudentID</th><th>StudentName</th><th>StandardID</th></tr><tr><td>1</td><td>John</td><td>1</td></tr><tr><td>2</td><td>Moin</td><td>1</td></tr></table>	StudentID	StudentName	StandardID	1	John	1	2	Moin	1
StudentID	StudentName	StandardID									
1	John	1									
2	Moin	1									
2	Standard 2	Key= 2 ▲ Grouping<Int32,Student> (2 items) ▶ <table><tr><th>StudentID</th><th>StudentName</th><th>StandardID</th></tr><tr><td>3</td><td>Bill</td><td>2</td></tr><tr><td>4</td><td>Ram</td><td>2</td></tr></table>	StudentID	StudentName	StandardID	3	Bill	2	4	Ram	2
StudentID	StudentName	StandardID									
3	Bill	2									
4	Ram	2									
3	Standard 3	(0 items)									

Lấy số lượng của mỗi nhóm

```
var query = Products.GroupJoin(PurchaseDetails,  
    p => p.Product_ID,  
    d => d.Product_ID,  
    (p, group) => new {  
        p.Product_Name,  
        Count = group.Count()  
    }  
);
```

IOrderedQueryable<> (7 items)	
Product_Name	Count
Macbook 13"	1
Dell XPS 15	1
Asus G703	0
Logitech G507	3
Apple Magic Mouse	2
Logitech K380	0
Razer Death 15X	0
	7

Đếm số lượng các sản phẩm bán chạy

Đếm số lượng các sản phẩm bán từ ngày đến ngày

```
var endDate = new DateTime(2020, 11, 25);
var startDate = endDate.AddDays(-7);

var details = Purchases.Where( p=> (startDate <= p.Created_At) && (p.Created_At <= endDate))
    .Join(PurchaseDetails,
        p => p.Purchase_ID,
        d => d.Purchase_ID,
        (p, d) => new {
            p.Created_At,
            Detail = d
        }
    )
    .Join(Products,
        d => d.Detail.Product_ID,
        p => p.Product_ID,
        (d, p) => new {
            d.Created_At,
            p.Product_ID,
            p.Product_Name,
            d.Detail.Quantity,
        }
    );
```

IOrderedQueryable<> (5 items)			
Created_At	Product_ID	Product_Name	Quantity
18/11/2020 12:00:00 AM	4	Logitech G507	15
19/11/2020 12:00:00 AM	4	Logitech G507	1324
19/11/2020 12:00:00 AM	5	Apple Magic Mouse	48
22/11/2020 12:00:00 AM	4	Logitech G507	2343
23/11/2020 12:00:00 AM	5	Apple Magic Mouse	2
			3732

Left join

```
var query = Foo.GroupJoin(  
    Bar,  
    foo => foo.Foo_Id,  
    bar => bar.Foo_Id,  
    (x,y) => new { Foo = x, Bars = y })  
.SelectMany(  
    x => x.Bars.DefaultIfEmpty(),  
    (x,y) => new { Foo=x.Foo, Bar=y});
```